EARTHLENS AI

Satellite Intelligence Platform

A Technical Project Dossier

Author : Gourgopal Mohapatra

Program : Artificial Intelligence & Data Science

GitHub : github.com/GOURGOPAL618

Contact : g.mohapatra.info.753@gmail.com

Version : 1.0

Date : 2026

1. EXECUTIVE SUMMARY

EarthLens AI is an end-to-end satellite data analysis platform built entirely in Python, designed to bridge the gap between raw Earth observation data and actionable geospatial intelligence. It connects to live satellite data sources — primarily Google Earth Engine, Sentinel Hub, and the USGS M2M API — processes multi-spectral imagery using established remote sensing indices, applies machine learning classification, and delivers everything through a clean, interactive web dashboard built on Streamlit.

The platform was built with a single core philosophy: satellite data should not require expensive GIS software, deep domain expertise, or manual data downloads to be useful. A researcher, a student, an environmental scientist, or a policy analyst should be able to open a browser, define a region of interest, set a date range, and receive meaningful geospatial insights within minutes — without writing a single line of code.

What EarthLens AI Does:

At its core, EarthLens AI is a spectral intelligence engine. Every satellite image is a stack of electromagnetic measurements across multiple wavelengths — visible light, near-infrared, shortwave infrared. These bands, when combined mathematically, reveal information invisible to the human eye: the health of a forest canopy, the extent of a flood, the spread of a wildfire, the growth of a city. EarthLens AI automates this entire process — from raw band ingestion to classified output map — across six specialized analysis modules.

Vegetation health is quantified through NDVI computation and five-class classification, distinguishing dense tropical forest from sparse grassland and bare soil. Water bodies are detected using a dual-index masking approach combining NDWI and MNDWI, reducing urban false positives that plague single-index methods. Land cover is classified using a machine learning pipeline — Random Forest, Decision Tree, and Logistic Regression — trained on eleven engineered spectral features per pixel. Multi-temporal change detection identifies deforestation, urbanization, and flooding events by analyzing NDVI differencing and Change Vector Analysis across two time periods. Burn area severity is mapped using the dNBR index

following the USGS classification standard, enabling post-fire damage assessment. Urban expansion is tracked using three complementary built-up indices — NDBI, UI, and IBI — to detect new construction, vegetation loss, and city densification over time.

Architecture and Data

The platform is architected in four clean layers: a data pipeline layer that handles APIauthentication and imagery fetching, a preprocessing engine for band normalization and cloud masking, an analysis and intelligence layer for spectral computation and ML classification, and a visualization hub that renders interactive Folium maps and Plotly statistical charts.

EarthLens AI supports five data sources spanning two satellites — Sentinel-2 at 10 meter resolution and Landsat 8/9 at 30 meter resolution — plus a fully functional offline synthetic mode that generates realistic Sentinel-2 band data locally, allowing complete development and testing without any API credentials.

Significance

Earth observation data is growing at an unprecedented rate. The Copernicus programme alone generates over 20 terabytes of satellite data daily. Yet the infrastructure to analyze this data at scale remains locked behind expensive licenses, complex workflows, and institutional access. EarthLens AI represents a step toward democratizing this infrastructure — making spectral analysis, ML classification, and geospatial visualization accessible through a single open-source Python platform.

The platform is fully open source under the MIT License and is designed not as a finished product, but as a research foundation — a base on which flood mapping, agricultural monitoring, SAR analysis, and deep learning classification can be built in future versions.

2.PROBLEM STATEMENT

2.1 The Data Paradox

We live in the golden age of Earth observation. Since the launch of Landsat-1 in 1972, humanity has been continuously photographing its own planet from space. Today, that effort has scaled into a global constellation of satellites — ESA's Sentinel series, NASA-USGS Landsat, ISRO's Resourcesat and Cartosat, and a growing wave of commercial operators like Pixxel, Planet Labs, and Maxar — capturing the Earth's surface in extraordinary detail, across dozens of spectral bands, multiple times per day.

The Copernicus programme alone generates over 20 terabytes of satellite data every single day. The entire Landsat archive — stretching back over 50 years — is freely available to anyone with an internet connection. Google Earth Engine hosts over 70 petabytes of geospatial datasets, updated in near real-time.

The data exists. It is abundant. It is free.

And yet, the vast majority of people who could benefit from it — environmental researchers, urban planners, disaster response teams, agricultural scientists, students, journalists, policy makers — cannot use it.

This is the satellite data paradox: the most comprehensive planetary monitoring system ever built, largely inaccessible to the people who need it most.

2.2 The Barriers

The reasons for this paradox are structural, not technical.

Barrier 1 — Software Complexity

Traditional geospatial analysis requires industry-grade GIS software — ArcGIS, ERDAS Imagine, QGIS with specialized plugins. These tools have steep learning curves, require

weeks of training to use effectively, and in the case of ArcGIS, carry commercial license costs that put them beyond the reach of most individual researchers and institutions in developing countries.

Barrier 2 — Data Format Complexity

Raw satellite imagery does not arrive as a JPEG or PNG. It arrives as multi-band GeoTIFF files, HDF archives, or SAFE format packages — binary formats that require specialized libraries to open, georeference, and interpret. A researcher encountering raw Sentinel-2 data for the first time faces a folder of cryptically named files with no obvious path to a usable image.

Barrier 3 — Domain Knowledge Requirements

Even with the right software and correctly opened data, the analysis itself demands deep remote sensing expertise. Which bands to combine for vegetation analysis? What threshold separates water from shadow in an NDWI map? How does atmospheric correction affect NDVI values? How should cloud-contaminated pixels be masked? These are not trivial questions — they require years of domain-specific training.

Barrier 4 — Infrastructure Requirements

Processing satellite imagery at regional or national scale requires significant computational infrastructure. A single Sentinel-2 tile covering 100 × 100 km contains over 100 million pixels across 13 bands. Running classification algorithms on this data locally demands high-memory machines and optimized geospatial computing pipelines — resources unavailable to most individual researchers.

Barrier 5 — Fragmented Tooling

Even for technically proficient users, the existing open-source ecosystem is fragmented. Rasterio handles band I/O. GDAL handles format conversion. scikit-learn handles classification. Folium handles visualization. Google Earth

Engine requires its own JavaScript or Python API. Stitching these tools into a coherent, end-to-end workflow is a significant engineering effort that most researchers — whose primary expertise is not software development — cannot afford to invest.

2.3 The Real-World Cost

These barriers have measurable consequences.

Deforestation monitoring in critical biodiversity zones is delayed because local environmental agencies lack the tools to process Sentinel-2 imagery rapidly. Flood extent mapping during disaster response relies on manual GIS analysis, taking hours when decisions need to be made in minutes. Agricultural yield prediction systems in India — where 60% of the population depends on farming — remain underdeveloped because satellite-based crop monitoring tools are inaccessible to state agriculture departments. Urban heat island mapping, wildfire damage assessment, glacier retreat monitoring, coastal erosion tracking — all of these are scientifically understood problems with clear satellite data solutions, bottlenecked not by data availability but by tooling accessibility.

In India specifically, where ISRO operates one of the world's most advanced Earth observation programmes, the gap between satellite data production and satellite data utilization remains vast. The data infrastructure exists. The analytical infrastructure does not.

2.4 What EarthLens AI Solves

EarthLens AI directly addresses each of these barriers.

Software complexity is eliminated by wrapping the entire analysis pipeline — data fetching, preprocessing, spectral computation, ML classification, and visualization — inside a

Streamlit web interface requiring no GIS expertise to operate.

Data format complexity is handled internally by the preprocessing engine, which manages band loading, normalization, cloud masking, and GeoTIFF I/O transparently.

Domain knowledge requirements are embedded into the platform itself — the choice of indices, the classification thresholds, the USGS severity standards, the dual-index water masking approach — so users receive expert-calibrated outputs without needing to be remote sensing experts.

Infrastructure requirements are addressed through Google Earth Engine integration, which offloads all heavy computation to Google's planetary-scale geospatial cloud, and through an offline synthetic mode that makes the full platform functional on any standard laptop.

Fragmented tooling is replaced by a single, coherent platform — one codebase, one interface, one workflow — from raw satellite bands to classified interactive map.

EarthLens AI does not replace the expertise of a trained remote sensing scientist. It removes the barrier that prevents everyone else from accessing satellite-based geospatial intelligence at all.

3.PLATFORM ARCHITECTURE

3.1 Design Philosophy

EarthLens AI is not a monolithic application. It is a layered, modular platform — each layer with a single, well-defined responsibility, communicating through clean interfaces, independently testable, and independently replaceable.

This design was driven by three principles.

Separation of concerns — the dashboard knows nothing about how bands are loaded. The analysis engine knows nothing about how maps are rendered. The data pipeline knows nothing about which ML model is running. Each layer does exactly one thing.

Source agnosticism — the analysis engine receives normalized NumPy arrays regardless of whether the data came from a synthetic generator, Google Earth Engine, Sentinel Hub, or USGS. Swapping a data source requires changing only the pipeline layer, nothing else.

Progressive complexity — a user with no API credentials can run the full platform in offline synthetic mode. A user with GEE authentication gets live Sentinel-2 imagery. A user with all APIs configured gets five data sources. The platform scales with the user's access level without any architectural change.

3.2 System Layers

EarthLens AI is built across four distinct architectural layers, each with clearly defined inputs, outputs, and responsibilities.

3.3 Data Flow — End to End

A complete analysis run inside EarthLens AI follows this exact sequence:

Step 1 — User Input The user opens the Streamlit dashboard, selects an analysis module from the sidebar, chooses a data source, sets a date range and bounding box, and clicks Run Analysis.

Step 2 — Data Acquisition The data pipeline layer receives the request parameters. Depending on the selected source, it either generates synthetic Sentinel-2 bands locally, authenticates with the GEE API and fetches a cloud-filtered composite, calls the Sentinel Hub API with an evalscript, or queries the USGS M2M API for a Landsat scene. The output in every case is a set of raw band arrays.

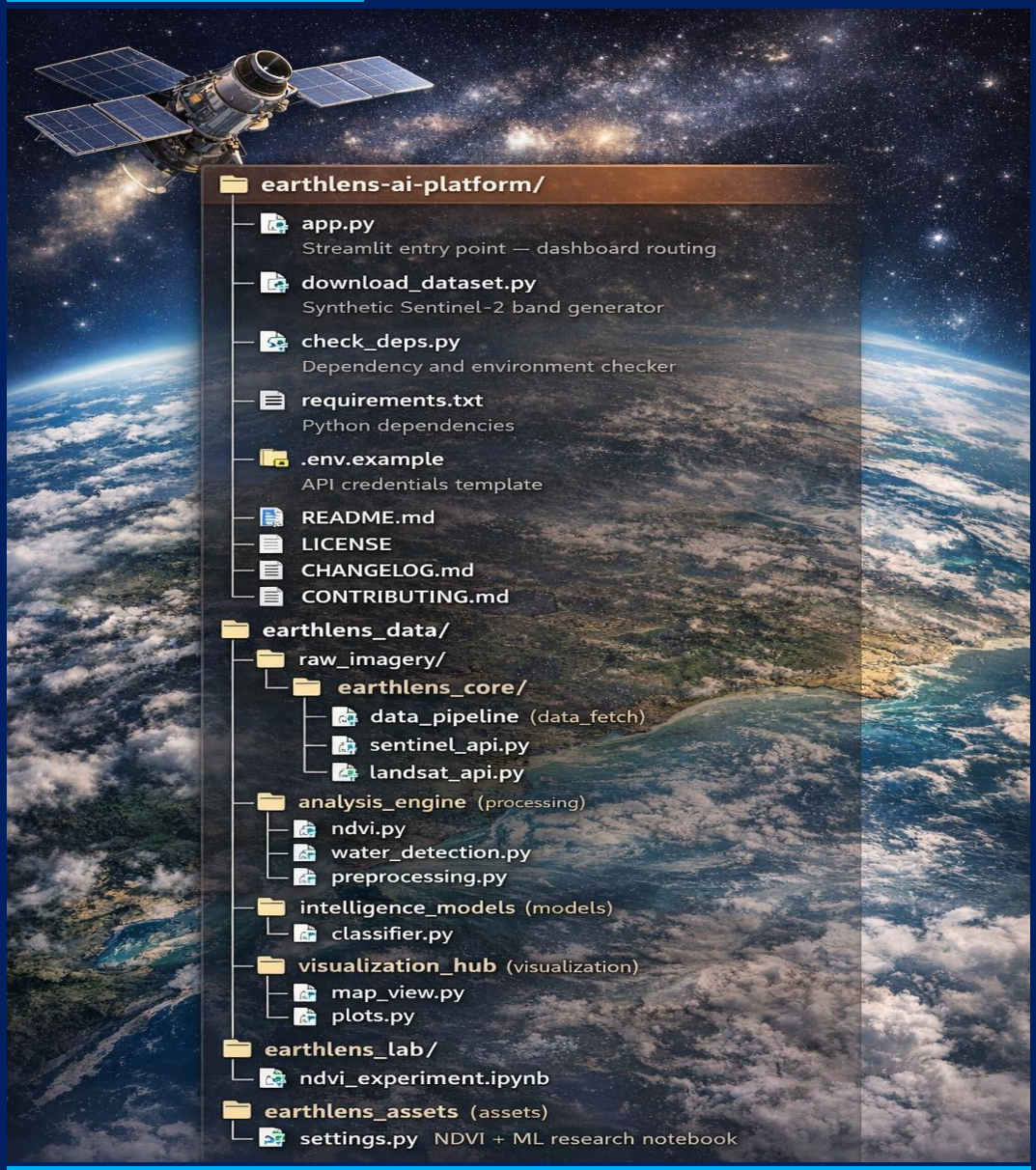
Step 3 — Preprocessing The preprocessing engine receives raw band arrays and performs normalization to the 0–1 range, invalid pixel masking, cloud pixel exclusion where cloud masks are available, and spatial resampling to align bands of different native resolutions — Sentinel-2 SWIR bands at 20m are resampled to match 10m optical bands.

Step 4 — Analysis The clean band stack is passed to the selected analysis module. Spectral indices are computed as vectorized NumPy operations across the full image array. For land use classification, feature engineering produces an 11-feature matrix, the ML model is trained on synthetic labels and applied to predict land cover class for every pixel. Statistical summaries — mean, standard deviation, per-class pixel counts — are computed alongside the spatial output.

Step 5 — Visualization Results are passed to the visualization hub. The classified array is colorized and overlaid on a Folium map centered on the analysis region. Statistical charts are generated in Plotly. Both are rendered inside the Streamlit interface.

Step 6 — Output The user sees an interactive map with layer controls, a statistical summary panel, and downloadable outputs — classified GeoTIFF and chart images saved to earthlens_data/processed_insights/.

3.4 Folder Structure

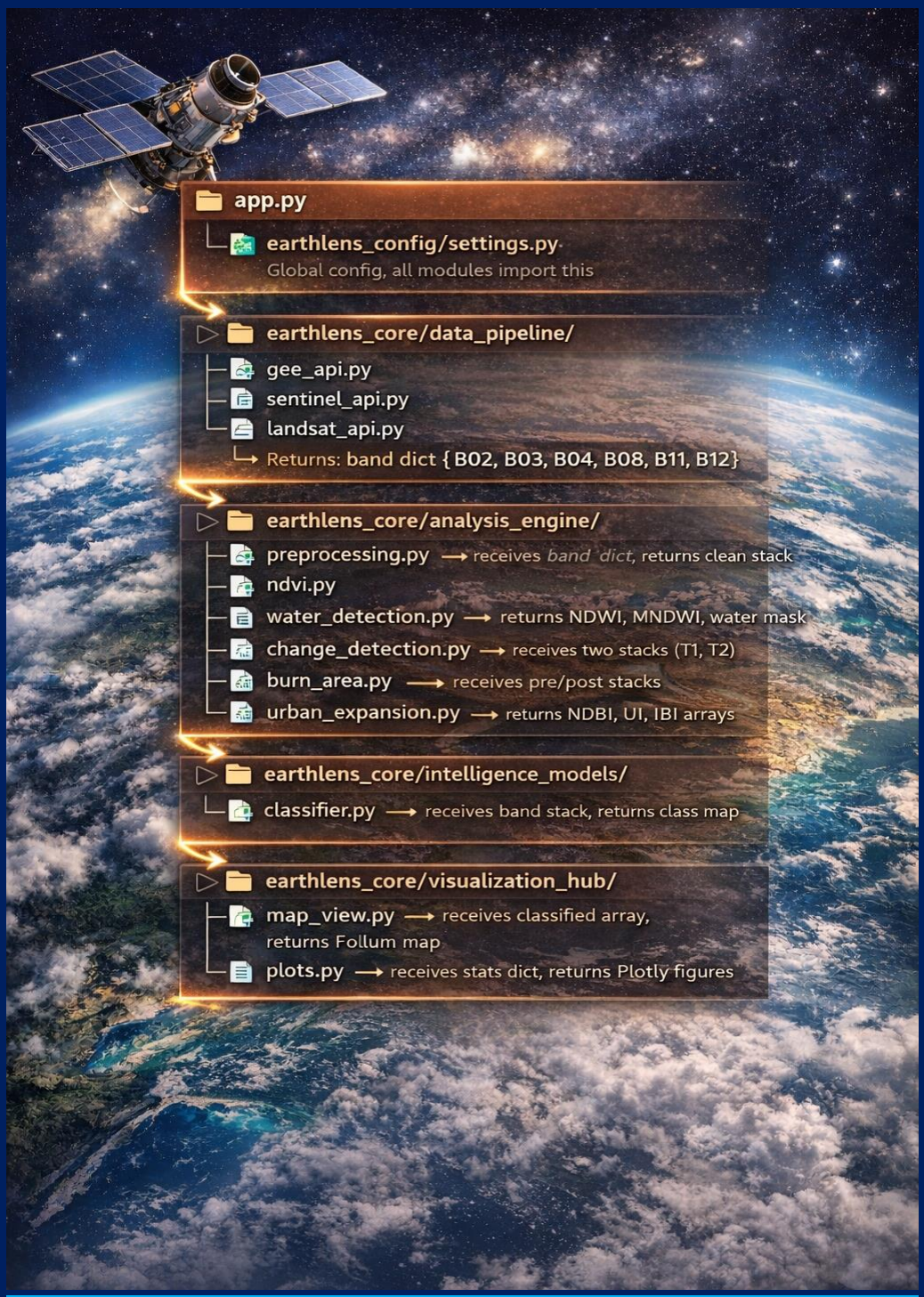


3.5 Technology Stack

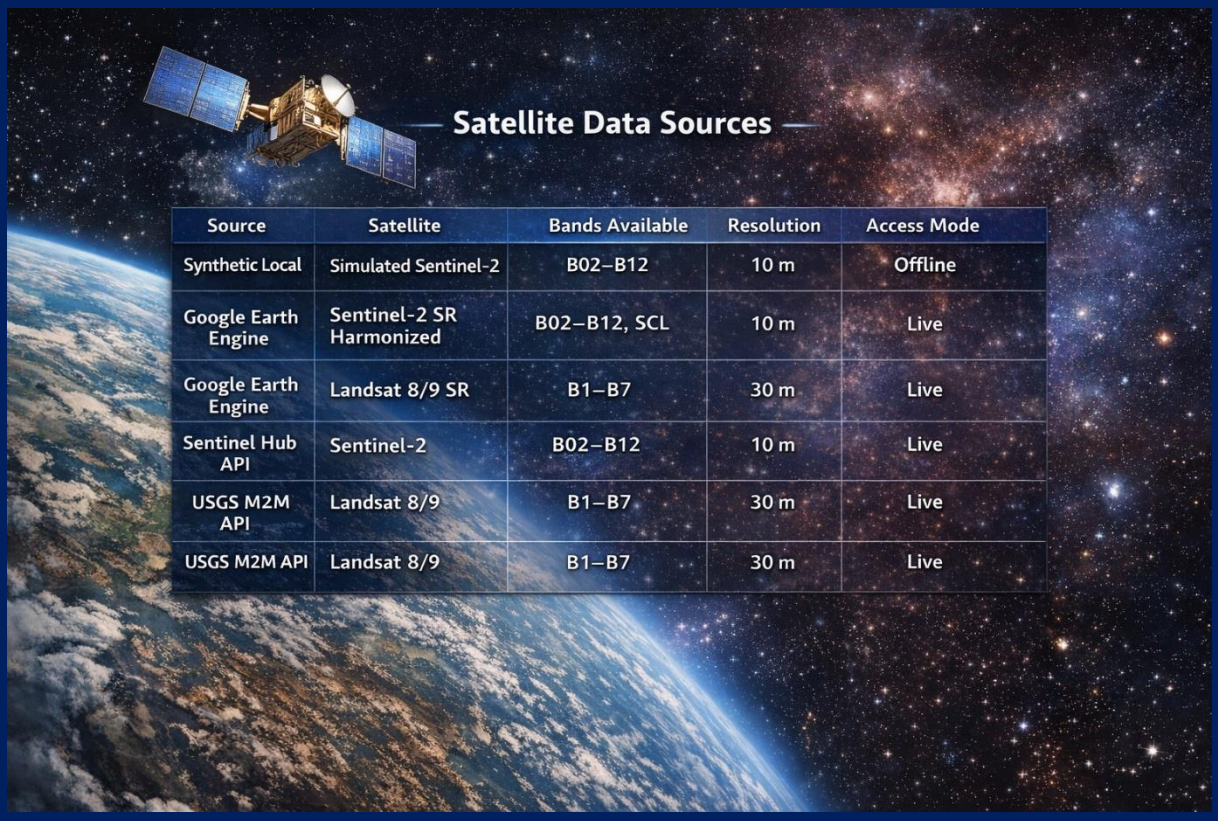
3.5 Technology Stack

Layer	Technology	Version	Purpose
Dashboard	Streamlit	≥ 1.32	Interactive web interface
Maps	Folium + streamlit-folium	≥ 0.16	Geospatial map rendering
Charts	Plotly	≥ 5.19	Statistical visualizations
Band I/O	Rasterio	≥ 1.3.9	GeoTIFF reading and writing
Numerical	NumPy	≥ 1.26	Vectorized array operations
Scientific	SciPy	≥ 1.12	Statistical computations
Tabular	Pandas	≥ 2.2	Results and metadata tables
ML	scikit-learn	≥ 1.4	Classification models
Model I/O	joblib	≥ 1.33	Model serialization
Satellite API	earthengine-api	≥ 0.1.390	GEE authentication and queries
GEE Helper	geemap	≥ 0.30	GEE Python utilities
HTTP	requests	≥ 2.31	Sentinel Hub and USGS API calls
Visualization	Matplotlib, Seaborn	≥ 3.8	Classification overlays
Logging	Loguru	≥ 0.7	Structured application logging
Experimentation	Jupyter	—	Research notebook environment

3.6 Module Interaction Map



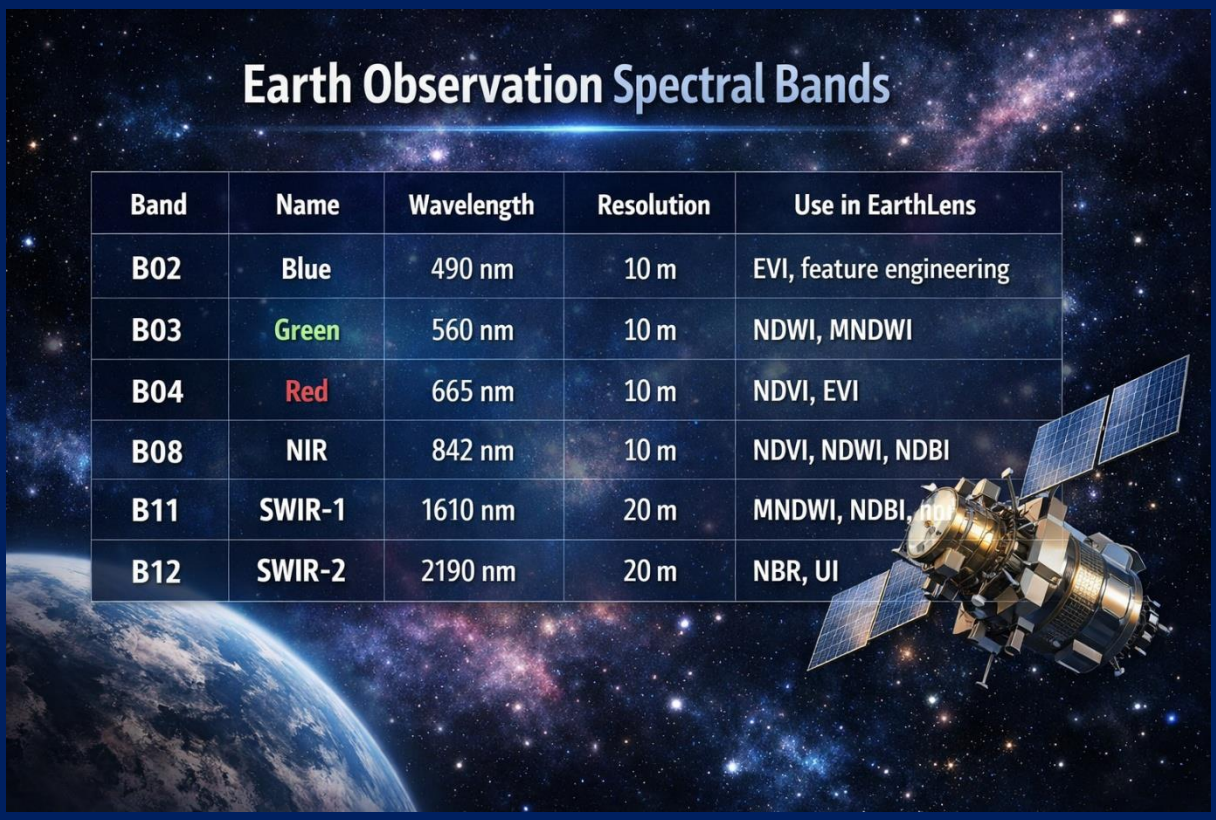
4. DATA SOURCES



Satellite Data Sources

Source	Satellite	Bands Available	Resolution	Access Mode
Synthetic Local	Simulated Sentinel-2	B02–B12	10 m	Offline
Google Earth Engine	Sentinel-2 SR Harmonized	B02–B12, SCL	10 m	Live
Google Earth Engine	Landsat 8/9 SR	B1–B7	30 m	Live
Sentinel Hub API	Sentinel-2	B02–B12	10 m	Live
USGS M2M API	Landsat 8/9	B1–B7	30 m	Live
USGS M2M API	Landsat 8/9	B1–B7	30 m	Live

🌍 Band Reference — Sentinel-2



Earth Observation Spectral Bands

Band	Name	Wavelength	Resolution	Use in EarthLens
B02	Blue	490 nm	10 m	EVI, feature engineering
B03	Green	560 nm	10 m	NDWI, MNDWI
B04	Red	665 nm	10 m	NDVI, EVI
B08	NIR	842 nm	10 m	NDVI, NDWI, NDBI
B11	SWIR-1	1610 nm	20 m	MNDWI, NDBI, nDI
B12	SWIR-2	2190 nm	20 m	NBR, UI

5. ANALYSIS MODULES

5.1 NDVI — Vegetation Health Mapping

FORMULA:

```
NDVI = (NIR - Red) / (NIR + Red)
     = (B08 - B04) / (B08 + B04)
```

What it does: NDVI quantifies vegetation density and health by exploiting the contrast between how plants reflect near-infrared light (strongly) and red light (weakly). Healthy vegetation absorbs red light for photosynthesis and reflects NIR, producing high NDVI values. Bare soil, urban surfaces, and water produce low or negative values.

Classification:

Class	NDVI Range	Interpretation
Water / Snow	−1.0 → 0.0	Open water, ice, snow cover
Bare / Urban	0.0 → 0.2	Built-up areas, bare soil, rock
Sparse Vegetation	0.2 → 0.4	Grassland, shrubs, degraded land
Moderate Vegetation	0.4 → 0.6	Mixed forest, agricultural land
Dense Vegetation	0.6 → 1.0	Tropical forest, healthy crops

Output: Classified raster map + per-class pixel statistics + Plotly histogram.

5.2 Water Detection

Formulas:

```
NDWI = (Green - NIR) / (Green + NIR)
= (B03 - B08) / (B03 + B08)
MNDWI = (Green - SWIR1) / (Green + SWIR1)
= (B03 - B11) / (B03 + B11)
```

What it does: NDWI identifies open water bodies but is sensitive to urban noise — built-up areas can produce false positives. MNDWI replaces NIR with SWIR-1, which suppresses urban signal and is more accurate for detecting water within or near cities. EarthLens applies a dual-index mask: a pixel is classified as water only when both NDWI > 0 and MNDWI > 0, significantly reducing false detections.

Output: Binary water mask + water coverage percentage + interactive Folium map overlay.

5.3 Land Use Classification

Feature Engineering — 11 features per pixel:

```
Raw Bands : B02 (Blue), B03 (Green),
B04 (Red), B08 (NIR), B11 (SWIR1), B12
(SWIR2)
Indices : NDVI, NDWI, MNDWI, NDBI,
EVI EVI Formula: EVI = 2.5 × (NIR - Red) /
(NIR + 6×Red - 7.5×Blue + 1)
NDBI Formula: NDBI = (SWIR1 - NIR) /
(SWIR1 + NIR) = (B11 - B08) / (B11 + B08)
```

- Models:

Model	Parameters	Strength	Target Classes
Random Forest	100 trees, default depth	Highest accuracy, handles non-linearity	Water·Dense Vegetation · Moderate Vegetation · Sparse Vegetation · Bare Soil · Urban / Built-up
Decision Tree	Max depth 10	Fast, interpretable, visualizable	Water·Dense Vegetation· Moderate Vegetation · Sparse Vegetation · Bare Soil · Urban / Built-up
Logistic Regression	L2 regularization	Baseline model, probabilistic output	Water · Dense Vegetation·Moderate Vegetation·Sparse Vegetation.Bare Soil · Urban / Built-up

Output: Classified land cover map + confusion matrix + per-class accuracy + model comparison table.

5.4 Change Detection

Method 1 — NDVI Differencing:

```
ΔV = NDVI_T2 - NDVI_T1

ΔV < -0.2 → Significant vegetation loss (deforestation, drought)

ΔV > +0.2 → Significant vegetation gain (regrowth, new crops)

Method 2 — Change Vector Analysis (CVA):

Magnitude = √[(NIR_T2 - NIR_T1)² + (Red_T2 - Red_T1)²]

Direction = arctan[(Red_T2 - Red_T1) / (NIR_T2 - NIR_T1)]
```

CVA captures both the magnitude and direction of spectral change, enabling differentiation between change types — deforestation, urbanization, and flooding produce distinct directional signatures in NIR-Red feature space.

Simulated change types: Deforestation · Urbanization · Flooding
Output: Change magnitude map + change type classification + before/after comparison.

5.5 Burn Area Detection Formulas:

```
NBR = (NIR - SWIR2) / (NIR + SWIR2)
    = (B08 - B12) / (B08 + B12)
```

dNBR = NBR_prefire − NBR_postfire

What it does: Healthy vegetation has high NIR reflectance and low SWIR-2 reflectance, producing high NBR values. Fire-affected areas lose chlorophyll and expose char and ash, drastically reducing NIR and increasing SWIR-2, producing low NBR values. The difference between pre-fire and post-fire NBR (dNBR) directly measures fire severity.

USGS Severity Classification:

Severity Class	dNBR Range	Description
Unburned	< 0.10	No fire impact
Low Severity	0.10 – 0.27	Surface fire, minor damage
Moderate-Low	0.27 – 0.44	Partial canopy loss
Moderate-High	0.44 – 0.66	Significant canopy damage
High Severity	> 0.66	Complete stand replacement

Output: dNBR severity map + burned area statistics + USGS class distribution chart.

5.6 Urban Expansion

Formulas:

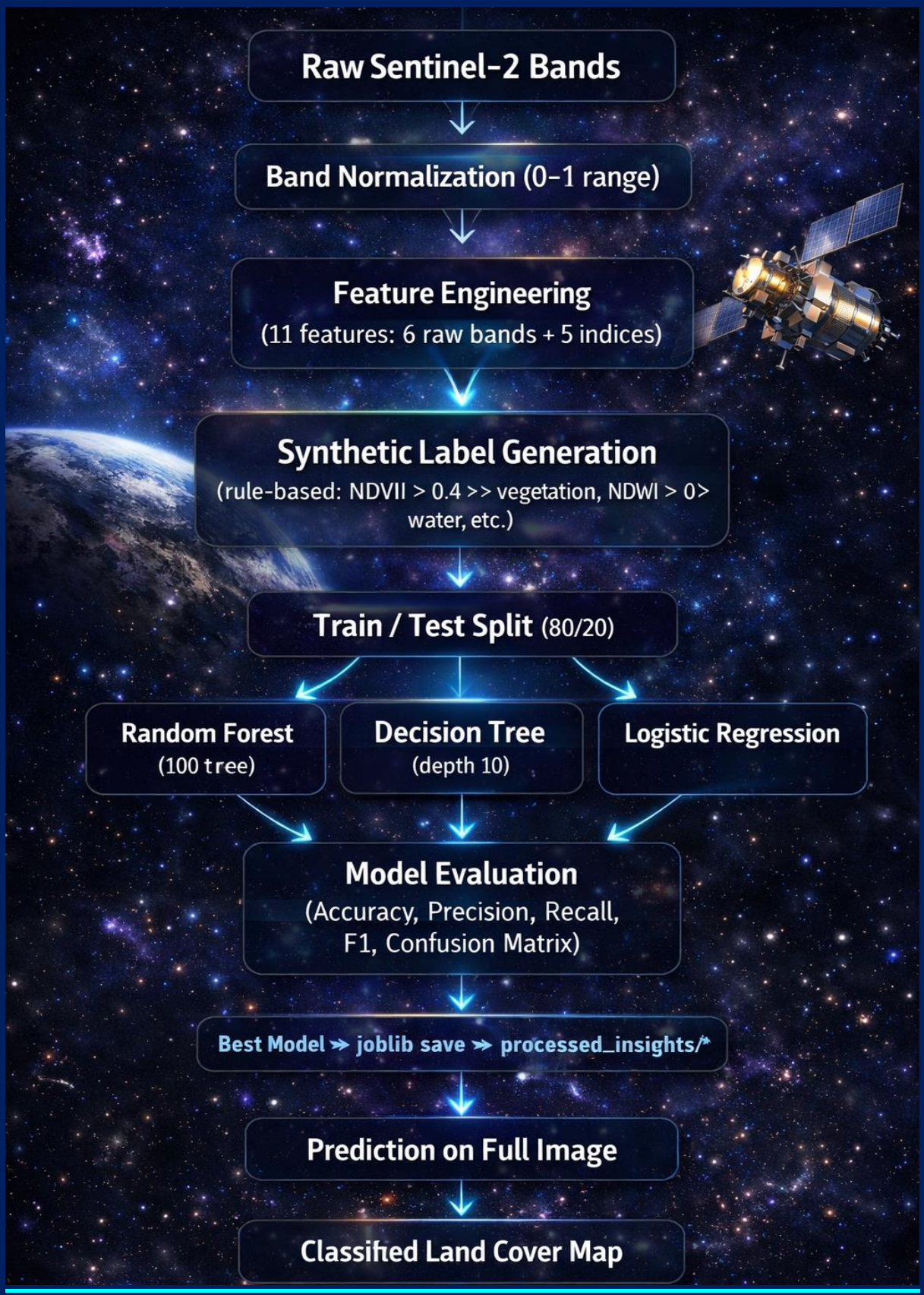
```
NDBI = (SWIR1 - NIR) / (SWIR1 + NIR) = (B11 - B08) / (B11 + B08)

UI = (SWIR2 - NIR) / (SWIR2 + NIR) = (B12 - B08) / (B12 + B08)

IBI = [NDBI - (NDVI + MNDWI)/2] / [NDBI + (NDVI + MNDWI)/2]
```

What it does: Built-up surfaces reflect SWIR strongly and NIR weakly — the opposite of vegetation. NDBI exploits this contrast. UI uses SWIR-2 instead of SWIR-1, capturing different built-up characteristics. IBI normalizes NDBI against vegetation (NDVI) and water (MNDWI) signals, producing the most accurate urban extraction of the three.

Multi-temporal analysis detects: new urban areas · vegetation-to-urban conversion · urban densification.

Output: Urban mask + expansion map + urbanization rate statistics.

6. MACHINE LEARNING PIPELINE



7. FUTURE ROADMAP

Version 1.0 — Current Six analysis modules fully operational GEE, Sentinel Hub, USGS API integration Streamlit dashboard with Folium and Plotly Offline synthetic mode for development

Version 1.5 — Planned Real GEE authentication flow inside the dashboard Flood extent mapping using SAR data (Sentinel-1) Agricultural monitoring module — crop health, irrigation detection Export functionality — download classified maps as GeoTIFF or PNG Region presets — major Indian cities, river basins, forest zones

Version 2.0 — Research Track Deep learning classification — CNN-based pixel classification Time series analysis — multi-year NDVI trend detection Integration with ISRO's Bhuvan platform REST API — EarthLens as a microservice Research paper submission — NDVI + ML classification results

The outlined roadmap reflects a structured progression from a functional prototype to a scalable and research-oriented geospatial intelligence platform. Version 1.0 establishes a solid operational base with reliable data pipelines and visualization tools, while Version 1.5 focuses on expanding real-world applicability through enhanced data access, domain-specific modules, and user-centric features. This phased approach ensures that the system remains stable while continuously evolving to handle more complex environmental and analytical challenges.

Looking ahead, Version 2.0 positions EarthLens as an innovation-driven platform aligned with modern advancements in AI and remote sensing. By integrating deep learning models, time-series analysis, and platforms like Bhuvan from ISRO, the project moves toward high-impact research and scalable deployment. The introduction of REST APIs further enables interoperability, allowing EarthLens to function as a microservice within larger ecosystems.

8. AUTHOR PROFILE

Gourgopal Mohapatra

B.Tech — Artificial Intelligence & Data Science

An AI and Space Technology researcher focused on the intersection of machine learning and Earth observation systems. EarthLens AI represents a hands-on deep dive into remote sensing, geospatial data pipelines, and satellite-based environmental monitoring — built from scratch with real data sources and production-grade architecture.

Domains: Remote Sensing · Geospatial AI · Satellite Data Analysis · Machine Learning

Target Sector: Indian Space Technology — Pixxel · GalaxEye · SatSure · KaleidEO · Digantara · ISRO

GitHub: github.com/GOURGOPAL618

Other Projects:

AstroVital AI — Clinical Decision Support System for astronaut health prediction under the 24-minute Mars communication delay. World's first CDSS of its kind targeting deep space crew autonomy.

EarthLens AI — Because every pixel tells a story. Built with 🛰 and Python.

THANK YOU